



EXTRAÇÃO DE DADOS POR BIG DATA E CONVERGÊNCIA COM IA

Prof. Dr. Julio César Santos dos Anjos
Dados e Informação

Técnicas de Big Data e Convergência com IA

— 23 de maio de 2025

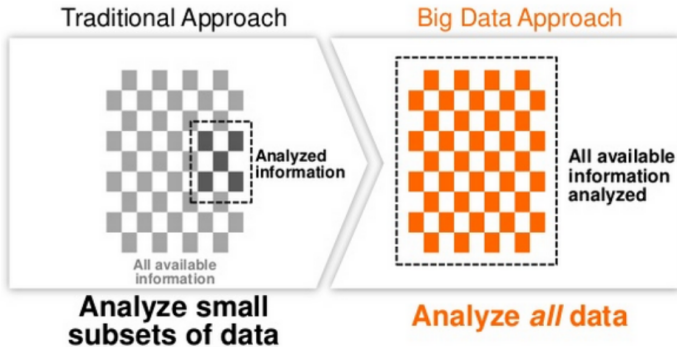
1. Abordagens para Extração com Big Data
2. Frameworks
3. Aplicações Típicas
4. Custos de Computação
5. Introdução da Programação MapReduce com Hadoop
6. Introdução à IA
7. Mecanismo de Computação
8. Adicionando Valor com IA



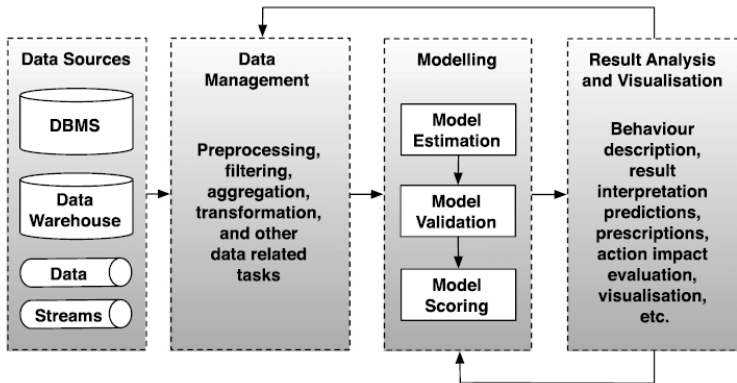
1. Abordagens para Extração com Big Data
2. Frameworks
3. Aplicações Típicas
4. Custos de Computação
5. Introdução da Programação MapReduce com Hadoop
6. Introdução à IA
7. Mecanismo de Computação
8. Adicionando Valor com IA



Abordagem Tradicional vs. Big Data:



Data Analytics Workflow



Fonte: [Assuncao et al. 2015]



1. Abordagens para Extração com Big Data
2. **Frameworks**
3. Aplicações Típicas
4. Custos de Computação
5. Introdução da Programação MapReduce com Hadoop
6. Introdução à IA
7. Mecanismo de Computação
8. Adicionando Valor com IA



MapReduce - Um *Framework* para Programação Paralela

- ✓ **Abstrai problemas de paralelização**, particionamento de dados, sincronização e comunicação.
- ✓ Programador deve definir somente 2 funções **Map** e **Reduce**.
- ✓ **Dados** são divididos e **transformados** em *tuplas* (chave, valor).
- ✓ Armazenamento é feito em **sistema de arquivos distribuídos**.
- ✓ **Utilizado em grandes data centers** como: Google, Amazon, FaceBook, Twitter, Yahoo, IBM, Microsoft, etc.

[Dean e Ghemawat 2004], [Venner 2009], [Dean e Ghemawat 2010]



MapReduce

- ✓ Desenvolvido em 2004 pelo Google;
- ✓ Implementação de código proprietário;
- ✓ É uma execução batch de logs de históricos de acesso à internet
 - Acesso de Internet baseados em nossas buscas de sites.

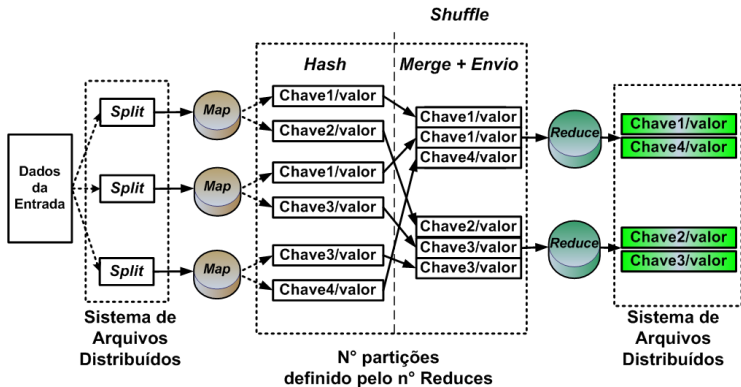
Apache Hadoop

- ✓ É um Open Source que segue o modelo MapReduce;
- ✓ A Implementação mais próxima do Google (conforme Jeff Dean*);
- ✓ Formado por diversos subprojetos;

*[Dean e Ghemawat 2004], [Venner 2009], [White 2012]



MapReduce Data Flow



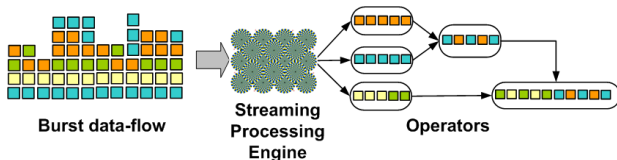
+ Tratamento de Falhas + Mecanismos Controle/Desempenho

Fonte: [Dean e Ghemawat 2010], [Lakshman e Malik 2010], [White 2012]

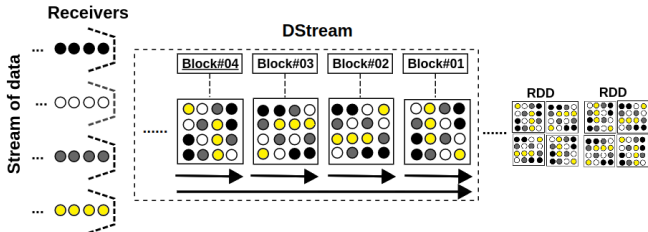


Streaming Processing

- ✓ Streaming Processing é um processamento orientado a eventos;
- ✓ Características:
 - É um fluxo contínuo de pequenos pacotes de dados;
 - Alto número de transações por segundo;
 - Modo de transmissão imprevisível em rajadas (Burst mode);
- ✓ Execuções near real-time (pelo menos $90\% < 1\text{ s}$);
- ✓ Transformação de dados ocorrem sobre operadores;
- ✓ Abordagem estatística.



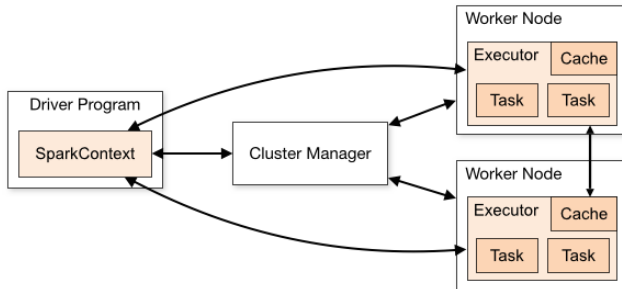
- ✓ Spark provém suporte ao processamento de streaming;
- ✓ Diferente da execução do Hadoop, seu processamento ocorre todo em memória;
- ✓ A abstração RDD (Resilient Distributed Datasets) permite o uso de armazenamento em memória (DStream);



- ✓ A Transformação dos dados são aplicadas nos RDDs através de operadores (FlatMap, Reduce, Join, Orderby, Merge, etc).



- ✓ Spark roda seus processos em um cluster de máquinas coordenados pelo SparkContext em seu controlador de programas (driver program);
- ✓ Um módulo Cluster Manager orquestra as operações entre o master e os workers;
- ✓ Os executores rodam tarefas nos workers que processam efetivamente as transformações de dados.



1. Abordagens para Extração com Big Data
2. Frameworks
- 3. Aplicações Típicas**
4. Custos de Computação
5. Introdução da Programação MapReduce com Hadoop
6. Introdução à IA
7. Mecanismo de Computação
8. Adicionando Valor com IA



Operações típicas do MapReduce:

1. Multiplicações **matriz-matriz** or **matriz-vetor**

Example: PageRank (uma computação de séries de Markov).

$$\vec{v}_i = \sum_{j=1}^n m_{ij} \cdot \vec{v}_j$$

2. Operações da Algebra Relacional

- ✓ Seleção;
- ✓ Projeção;
- ✓ União, intersecção e diferença;
- ✓ Junção (join);
- ✓ Agregação e Agrupamento (clustering).

Fonte: [Leskovec, Rajaraman e Ullman 2020]



Seleção:

- ✓ Seja uma condição C aplicada a cada tupla na relação R ;
→ Saída é uma tupla $\xi \xrightarrow{C} (R)$
- ✓ **Map** faz a seleção sozinho;
→ Saída é uma tupla $\xi \xrightarrow{C} (R)$;
→ Para cada tupla t em R , testa a condição C . Se verdadeiro, produz uma chave/valor (t, t) ;
- ✓ **Reduce** é a identidade e simplesmente repassa o valor do Map para a saída.



Projeção:

- ✓ Seja um subconjunto S de atributos da relação R ;
 - Saída é uma tupla de componentes para os atributos em S onde $\pi \xrightarrow{S} (R)$
- ✓ **Map** faz a seleção da tupla t que pode aparecer várias vezes;
 - Cada entrada contém uma tupla t em R ;
 - O Map produz uma tupla t' por eliminar de t aqueles componentes $\notin S$ o resultado produz chave/valor (t', t') .
- ✓ **Reduce** recebe o uma entrada $(t', [t', t', \dots, t'])$, verifica se $t' \in S$ e remove duplicatas.
 - Saída é uma chave/valor (t', t') ;



União:

- ✓ Sejam duas relações $P \wedge S$ com esquemas semelhantes;
- ✓ **Map** indica tuplas t em $P \vee S$;
 - Produz uma saída chave/valor (t, t') não importando qual relação;
- ✓ **Reduce** recebe o valor do Map e emite uma chave/valor sempre que $t' \in P \vee t' \in S$ e remove duplicatas.
 - Saída é um conjunto de tuplas que reitem $\xi \xrightarrow{P} (R) \vee \xi \xrightarrow{S} (R)$;



Intersecção:

- ✓ Sejam duas relações $P \wedge S$ com esquemas semelhantes;
- ✓ **Map** indica tuplas t em $P \vee S$;
 - Produz uma saída chave/valor (t, t') não importando qual relação;
- ✓ **Reduce** recebe o valor do Map e produz uma chave somente se $t' \in P \wedge t' \in S$, removendo duplicatas;
 - Saída é um conjunto de tuplas que relem $\xi \xrightarrow{P} (R) \wedge \xi \xrightarrow{S} (R)$;



Diferença:

- ✓ Sejam duas relações $P \wedge S$ com esquemas semelhantes. O único modo de uma tupla $t \rightarrow (t, t')$ aparecer na saída $P - S$ é estar presente em P e não estar em S ;
- ✓ **Map** verifica se para cada entrada há uma tupla t em $P \vee S$
 - Produz chave/valor $(t, P) \vee (t, R)$ indicando o conjunto de origem;
- ✓ **Reduce** recebe o valor do Map e verifica a relação $t \in P \wedge t \notin S$ e remove duplicatas da relação;
 - Saída é um conjunto de tuplas (t, t) que reitem
 - $\exists \xi \xrightarrow{P} (R) \wedge \nexists \xi \xrightarrow{S} (R)$;



Natural Join (Associação):

- ✓ Seja uma Associação $P(A, B) \wedge S(B, C)$. Deve-se encontrar as tuplas para os componentes de A que relacionam-se com B em P e os componentes de C que se relacionam com o B em S ;
- ✓ **Map** verifica se para cada entrada há uma tupla t em $P \vee S$
 - Produz chave/valor $(b, (P(a)) \in P \wedge (b, S(c)) \in S$;
- ✓ **Reduce** recebe o valor do Map e para uma mesma chave b ;
 - Saída é um conjunto de tuplas $(b, (a, c))$ que reletem a relação;



Agrupamento e Agregação:

- ✓ Dada uma relação $R(A, B, C)$ particiona as tuplas em um conjunto de atributos denotado por $\varphi_{A, \theta(B)}(R)$;
- ✓ **Map** produz chaves (a, b) para cada entrada que contém uma tupla t em R
 - A saída é uma lista $(a, [b_1, b_2, \dots, b_n])$
- ✓ **Reduce** recebe o valor do Map e produz (a, θ) onde $\theta = \sum_{k=1}^n b_k$;
 - Saída é um conjunto de tuplas (a, θ) que relembram R ;



Aplicações Big Data

- ✓ Várias domínios possíveis;
- ✓ Forte intersecção com aplicações de IA;



Retail



Health Care



Mobility



Information Technology



Risk Management



Data Analytics



Education



Research

1. Abordagens para Extração com Big Data
2. Frameworks
3. Aplicações Típicas
- 4. Custos de Computação**
5. Introdução da Programação MapReduce com Hadoop
6. Introdução à IA
7. Mecanismo de Computação
8. Adicionando Valor com IA



Relação de Custos

- ✓ Cada tarefa executa inteiramente em um nó;
 - Tempo tende a ser linear com a entrada de dados;

- ✓ Latência da Carga de dados na memória
 - A latência da leitura dos dados do disco para memória pode ser muito maior que o tempo de execução da própria tarefa;

- ✓ Custo de comunicação é dominante;
 - A saída do algoritmo é raramente maior se comparada com a entrada + dados intermediários;



Cálculo do Custo de Comunicação

- ✓ Tomando-se o join entre dois datasets $R(A, B) \bowtie S(B, C)$ e o custo de comunicação das relações são r e s respectivamente;
→ $R \bowtie S = \{t \cup z | t \in R \wedge z \in S \wedge f(t \cup z)\}$
- ✓ A soma do custo de comunicação do Map é $r + s$
→ Cada saída do Map é enviada para uma única tarefa Reduce através da rede;
- ✓ A comunicação tem um custo $O(r + s)$;
→ O custo da execução da tarefa de redução no Reduce depende dos dados intermediários do Map;



Complexidade do Modelo MapReduce - Condições

- ✓ Seja o **tamanho do redutor** (q) e a **taxa de replicação** (r) parâmetros que caracterizam o algoritmo do MapReduce.

Tamanho do Redutor

- ✓ O tamanho do redutor (q) é o **limite superior** do número de valores que podem aparecer em uma lista de partições associada a uma única chave.
 - Já o tamanho do redutor pode ser selecionado com as seguintes estratégias:
 - Tornar o tamanho do redutor pequeno (força muitos reduces);
 - Escolher um **tamanho de redutor para ser executado** inteiramente na **memória principal** da máquina;



Complexidade do Modelo MapReduce - Condições

Taxa de Replicação

- ✓ A taxa de replicação (r), representa o número de pares {chave,valor} produzido por todas as tarefas Map divididas pelo número de entradas;
 - taxa de replicação representa a comunicação média de tarefas de uma tarefa Map para um Reduce.
- ✓ Dados estes requisitos a complexidade estará relacionada com o tipo de aplicação.



Ex.:Complexidade para a Multiplicação de Matrizes

- ✓ Considere uma multiplicação de matrizes de $n \times n$ elementos
 - A taxa de replicação r é igual a n ;
 - Para cada m_{ij} há n elementos em cada matriz;
 - Cada entrada produz um número de pares (i, k) , $1 \leq k \leq n$;
- ✓ O tamanho de $q = n + n = 2n$;
- ✓ Neste caso q é tão pequeno quanto ele pode ser e r é seu **máximo**.
Assim:
 - O número de valores que aparecem em uma lista e a relação do número de pares chave/valor pode ser dada por:
 - $qr = 2n \times n$ para manter a coerência da análise $qr \geq 2n^2$.
 - Limite assintótico inferior $\Omega(n^2)$
Isto é melhor do que usar o algoritmo de multiplicação de matrizes recursiva $\Theta(n^3)$.



1. Abordagens para Extração com Big Data
2. Frameworks
3. Aplicações Típicas
4. Custos de Computação
- 5. Introdução da Programação MapReduce com Hadoop**
6. Introdução à IA
7. Mecanismo de Computação
8. Adicionando Valor com IA



Considerações sobre os dados a serem analisados:

Entrada:

Um conjunto de textos;

Objetivo:

Saber a quantidade de palavras existentes nestes textos;

Saída:

Uma lista de palavras com suas incidências nos textos;

Uso de APP de Programação chamada WordCount.



Exemplo de pseudo-código MapReduce para o Word Count

```

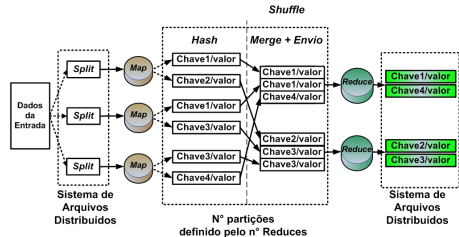
map (line_number, text):
  for each word in word_list: do
    emit (word, 1)
  end for

```

```

reduce (word, values[]):
  word_count = 0
  for each v in values: do
    word_count+ = v
  end for
  emit (word, word_count)

```



Exemplo de pseudo-código MapReduce para o Word Count

```
map (line_number, text):  
for each word in word_list: do  
    emit (word, 1)  
end for
```

```
reduce (word, values[]):  
word_count = 0  
for each v in values: do  
    word_count+ = v  
end for  
emit (word, word_count)
```

A função *Map* lê cada linha de uma entrada em formato texto e para cada palavra encontrada, emite uma chave intermediária (palavra,1)



Exemplo de pseudo-código MapReduce para o Word Count

```
map (line_number, text):  
for each word in word_list: do  
    emit (word, 1)  
end for
```

```
reduce (word, values[]):  
word_count = 0  
for each v in values: do  
    word_count+ = v  
end for  
emit (word, word_count)
```

A função *Map* lê cada linha de uma entrada em formato texto e para cada palavra encontrada, **emite uma chave intermediária (palavra,1)**



Exemplo de pseudo-código MapReduce para o Word Count

```
map (line_number, text):  
for each word in word_list: do  
    emit (word, 1)  
end for
```

```
reduce (word, values[]):  
    word_count = 0  
for each v in values: do  
    word_count+ = v  
end for  
    emit (word, word_count)
```

A função *Reduce* “soma” as
chaves de mesmo índice e emite
nova chave (palavra, valor)



Exemplo de pseudo-código MapReduce para o Word Count

```
map (line_number, text):  
  for each word in word_list: do  
    emit (word, 1)  
end for
```

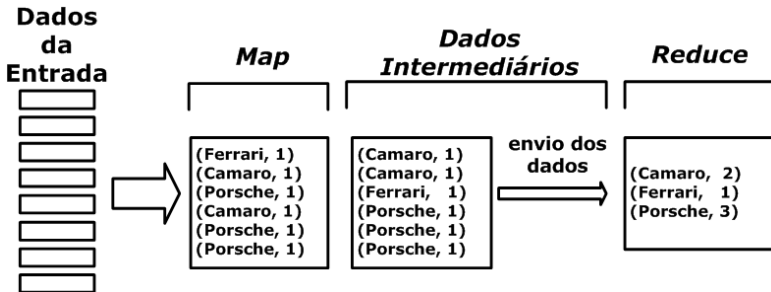
```
reduce (word, values[]):  
  word_count = 0  
  for each v in values: do  
    word_count+ = v  
  end for  
  emit (word, word_count)
```

A função *Reduce* “soma” as
chaves de mesmo índice e **emite**
nova chave (palavra, valor)

Operação da álgebra relacional de: **agregação**.



EXEMPLO WORD COUNT



Fases do Processamento do Word Count no MapReduce.

Modelo de Entrada e Saída de Dados

map : $(k_1, v_1) \rightarrow \text{list}(k_2, v_2) \rightarrow \text{combine} : (k_2, \text{list}(v_2))$

reduce : $(k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$

Classe Mapper

Class Mapper<KEY_{in}, VALUE_{in}, KEY_{out}, VALUE_{out}>

Classe Reducer

Class Reducer<KEY_{in}, VALUE_{in}, KEY_{out}, VALUE_{out}>

Pacotes do Hadoop

Mapreduce

Mapred



Tipos do Hadoop comparados com primitivas Java

Tipo Hadoop	Primitiva Java eq.	Tamanho (Bytes)
BooleanWritable	boolean	1
ByteWritable	byte	1
IntWritable	int	2
VIntWritable		1-5
FloatWritable	float	4
LongWritable	long	8
VLongWritable		1-9
DoubleWritable	double	8
Text	java.lang.String	2 GB



Traduzindo o exemplo do WordCount em um programa

Programação MapReduce estrutura básica

```
public class WordCount {  
  
    public static class Mapper []  
  
    public static class Reducer []  
  
    public static void main(String[] args) throws Exception {..  
}
```



Exemplo de Programação MapReduce - Imports

```
import java.io.IOException;  
import java.util.StringTokenizer;  
  
import org.apache.hadoop.conf.Configuration;  
import org.apache.hadoop.fs.Path;  
  
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.Text;  
  
import org.apache.hadoop.mapreduce.Job;  
import org.apache.hadoop.mapreduce.Mapper;  
import org.apache.hadoop.mapreduce.Reducer;  
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;  
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;  
  
import org.apache.hadoop.util.GenericOptionsParser;
```



Exemplo de Programação MapReduce - Classe Mapper

```
public static class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
        ) throws IOException, InterruptedException {
        StringTokenizer itr = new StringTokenizer(value.toString());

        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```



Exemplo de Programação MapReduce - Classe Reducer

```
public static class IntSumReducer
    extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context
        ) throws IOException, InterruptedException {
        int sum = 0;

        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```



Exemplo de Programação MapReduce - Definição da main

```
public static void main(String[] args) throws Exception {  
    Configuration conf = new Configuration();  
    String[] otherArgs = new GenericOptionsParser(conf, args).getRemainingArgs();  
  
    if (otherArgs.length < 2) {  
        System.err.println("Usage: wordcount <in> [<in>...] <out>");  
        System.exit(2);  
    }  
    Job job = Job.getInstance(conf, "word count");  
    job.setJarByClass(WordCount.class);  
    job.setMapperClass(TokenizerMapper.class);  
    ⋮  
}
```



Exemplo de Programação MapReduce - Definição da main

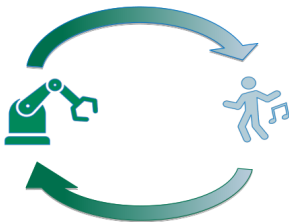
```
⋮  
⋮  
job.setCombinerClass(IntSumReducer.class);  
job.setReducerClass(IntSumReducer.class);  
job.setOutputKeyClass(Text.class);  
job.setOutputValueClass(IntWritable.class);  
for (int i = 0; i < otherArgs.length - 1; ++i) {  
    FileInputFormat.addInputPath(job, new Path(otherArgs[i]));  
}  
FileOutputFormat.setOutputPath(job,  
    new Path(otherArgs[otherArgs.length - 1]));  
System.exit(job.waitForCompletion(true) ? 0 : 1);  
}  
}
```



1. Abordagens para Extração com Big Data
2. Frameworks
3. Aplicações Típicas
4. Custos de Computação
5. Introdução da Programação MapReduce com Hadoop
- 6. Introdução à IA**
7. Mecanismo de Computação
8. Adicionando Valor com IA

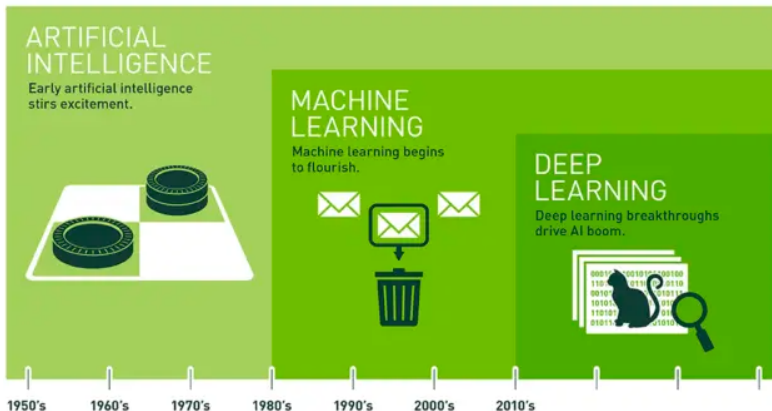


Humano vs Arpedizagem de Máquina



Humano	Máquina
Descansar e Digerir	Treinamento
Lutar ou Correr	Predição

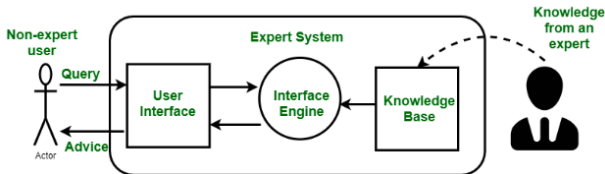
Evolução da IA ao longo dos tempos



Tipos de Programação

✓ Sistemas Especialistas ou Sistemas Baseados em Conhecimento

- Altamente complexos
- Depende de várias regras bem definidas (Conjunto de IFs)
- Programação envolve várias pessoas especialistas no tema
- Conhecimento detalhado dos processos e do Data Flow



Limitações de Sistemas Especialistas

O que são estas três imagens?



Limitações de Sistemas Especialistas

O que são estas três imagens?



Para nós pode ser fácil, mas criar regras para esta classificação é uma tarefa muito difícil.

Como nós aprendemos?

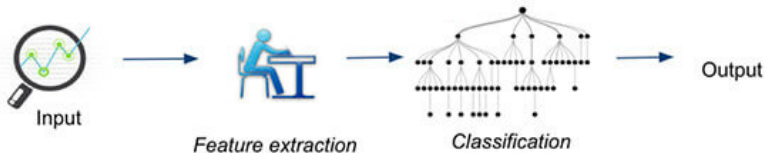


- ✓ Somos expostos a uma quantidade enorme de dados
- ✓ Vemos a resposta correta (labels)
- ✓ Identificamos parâmetros para gerar padrões por conta própria.

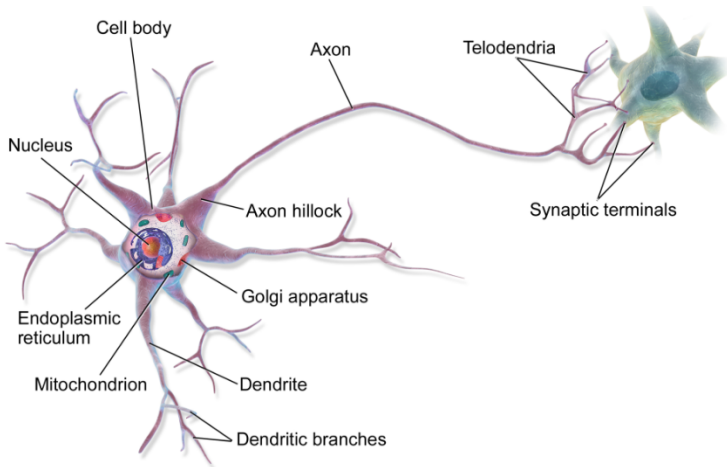
Tipos de Programação

✓ Algoritmos de Redes Neurais

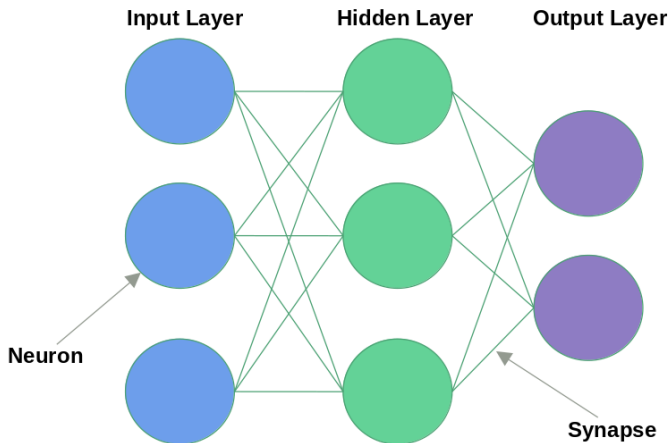
- Inspirado em modelos biológicos;
- Criada na década de 50;
- Identificam-se features que compõem o modelo
- Processado por máquinas Von Neumann (CPU only);
- Não é necessário o conhecimento detalhado dos processos mas dados devem ter uma estrutura;



Modelo inicial - Modelo Biológico



Modelo Computacional inspirado no Modelo Biológico



Features

- ✓ Features são observações que são utilizadas para formar a predição
 - Para a classificação de imagem, os pixels são as features;
 - Para o reconhecimento da voz, o tom e o volume do som são exemplos de features;
 - Para carros autônomos, dados das cameras, a variação dos sensores, e GPS são features.
- ✓ Extrair features relevantes é importante para construir um modelo
 - Hora do dia é uma feature irrelevante quando classifica-se imagens;
 - Hora do dia é uma feature relevante quando classifica-se emails porque os SPAMs muitas vezes ocorrem a noite.

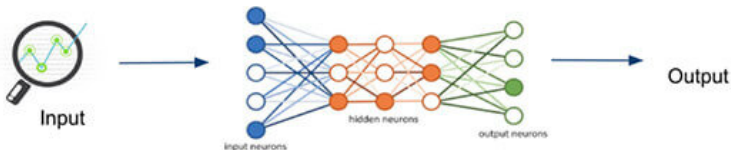


Tipos de Programação

✓ Deep Learning

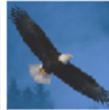
- Evolução a partir das redes neurais;
- São criados vários níveis de features (camadas ocultas);
- Processado por GPUs;
- Não é necessário o conhecimento dos processos ou dos dados;
- Mas se o modelo tiver um ajuste muito específico para um conjunto de dados, o modelo não generaliza para um domínio qualquer.

Ex.: Classificação de imagens de animais.



Redes Neurais profundas (Deep Learning)

- ✓ Para identificar padrões é necessário uma quantidade muito grande de dados;
 - Dados de um domínio e outros que não estão no domínio. Ex. gato, cachorro, lobo, etc
- ✓ A Internet pode ser uma fonte de dados inestimável;
Ex. Resultado da classificação da imagem de um gato.

Prediction:						
Image:						
	True Positive	True Negative	False Negative	False Positive		

Quando deve-se usar algoritmos de Deep Learning?

Tipo de Programação	Descrição
Clássica	Se as regras são claras e diretas muitas vezes é melhor usar este modelo.
Deep Learning	Se as regras são confusas, complexas e difíceis de definir, deve-se usar este modelo.

Deep Learning vs outros métodos de IA

- ✓ Redes profundas e complexas;
- ✓ Bilhões de parâmetros ou mais;
- ✓ Várias camadas em um modelo;
- ✓ Importante para aprendizagem de regras complexas.



Métricas Usuais

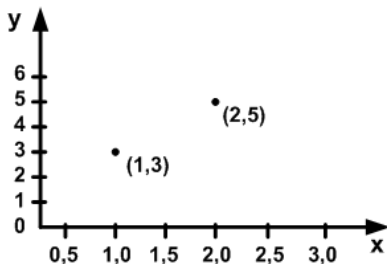
Métrica	Descrição	Cálculo
Precisão	Percentual de label positivos que estão corretos	$P = \frac{\# \text{verdadeiro positivos}}{\# \text{verdadeiro positivos} + \# \text{falso positivos}}$
Recall	Percentagem de casos positivos identificados corretamente	$R = \frac{\# \text{verdadeiro positivos}}{\# \text{verdadeiro positivos} + \# \text{falso negativos}}$
Acurácia	Percentagem de casos identificados corretamente	$A = \frac{\# \text{verdadeiro positivos} + \# \text{verdadeiro negativos}}{\# \text{amostras}}$



1. Abordagens para Extração com Big Data
2. Frameworks
3. Aplicações Típicas
4. Custos de Computação
5. Introdução da Programação MapReduce com Hadoop
6. Introdução à IA
- 7. Mecanismo de Computação**
8. Adicionando Valor com IA



Modelo de Computação



x	y
1	3
2	5

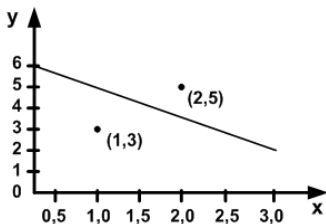
- ✓ Dado 2 pontos quaisquer, deseja-se encontrar uma função que passe por estes 2 pontos;
- ✓ O modelo adequado é a equação da reta.

$$y = \alpha x + \beta$$

- ✓ Tem-se que calcular o valor de α e β
- ✓ Um método, seria o de tentativa e erro. Atribui-se uma reta aleatória e calcula-se a diferença entre o valor computado e o desejado com objetivo de minimizar o erro.



Modelo de Computação



x	y
1	3
2	5

$$\alpha = -\frac{4}{3} \text{ e } \beta = 6$$

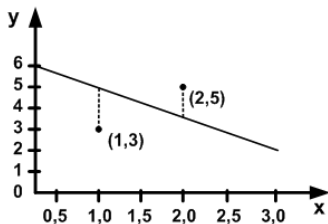
- ✓ Dado 2 pontos quaisquer, deseja-se encontrar uma função que passe por estes 2 pontos;
- ✓ O modelo adequado é a equação da reta.

$$y = \alpha x + \beta$$

- ✓ Tem-se que calcular o valor de α e β
- ✓ Um método, seria o de tentativa e erro. Atribui-se uma reta aleatória e calcula-se a diferença entre o valor computado e o desejado com objetivo de minimizar o erro.



Modelo de Computação



x	y	$\hat{y}$
1	3	4,67
2	5	3,33

$$\alpha = -\frac{4}{3} \text{ e } \beta = 6$$

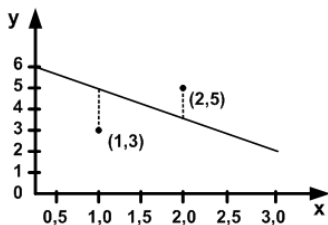
- ✓ Dado 2 pontos quaisquer, deseja-se encontrar uma função que passe por estes 2 pontos;
- ✓ O modelo adequado é a equação da reta.

$$y = \alpha x + \beta$$

- ✓ Tem-se que calcular o valor de α e β
- ✓ Um método, seria o de tentativa e erro. Atribui-se uma reta aleatória e calcula-se a diferença entre o valor computado e o desejado com objetivo de minimizar o erro.



Modelo de Computação



x	y	$\hat{y}$	$(y - \hat{y})^2$
1	3	4,67	2,7889
2	5	3,33	2,7889

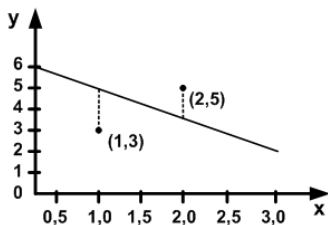
$$\alpha = -\frac{4}{3} \text{ e } \beta = 6$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2 = 2,7889$$

- ✓ MSE → encontra as diferenças médias quadradas entre os resultados esperados e previstos.
- ✓ O MSE é o segundo momento do erro. Ele incorpora tanto a variância do estimador e seu viés.
- ✓ Sensível a outliers.



Modelo de Computação



x	y	$\hat{y}$	$(y - \hat{y})^2$
1	3	4,67	2,7889
2	5	3,33	2,7889

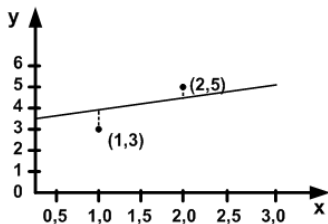
$$\alpha = -\frac{4}{3} \text{ e } \beta = 6$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2} = 1.67$$

- ✓ RMSE → é uma medida de precisão para comparar erros de previsão de diferentes modelos para um determinado conjunto de dados e não entre conjuntos de dados.
- ✓ Ele é dependente da escala.



Modelo de Computação



x	y	$\hat{y}$	$(y - \hat{y})^2$
1	3	4	1
2	5	4,5	0,25

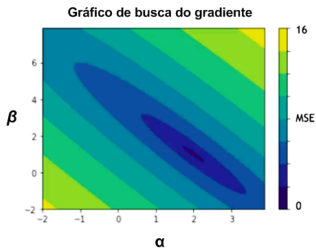
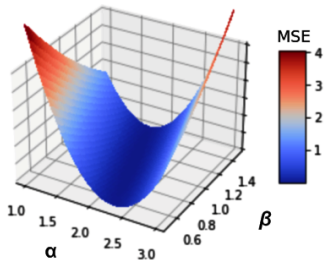
$$\alpha = 0,5 \text{ e } \beta = 3,5$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2} = 0,790$$

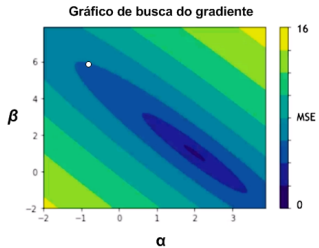
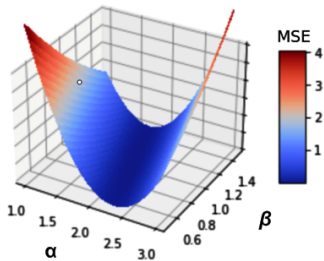
- ✓ RMSE → é uma medida de precisão para comparar erros de previsão de diferentes modelos para um determinado conjunto de dados e não entre conjuntos de dados.
- ✓ Ele é dependente da escala.



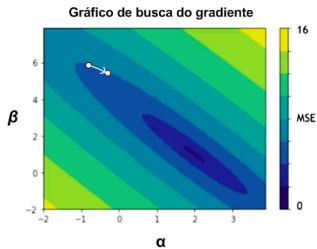
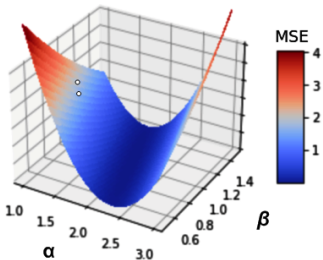
Busca de um erro mínimo (ou mínimo para a Função de Perda)



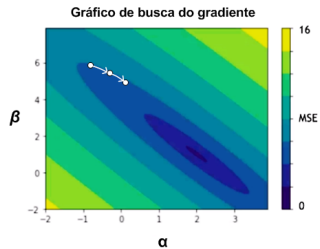
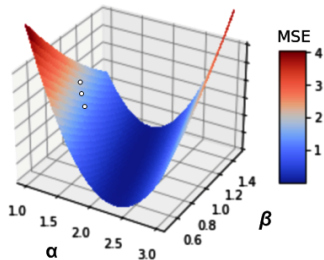
Busca de um erro mínimo (ou mínimo para a Função de Perda)



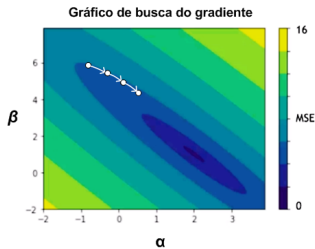
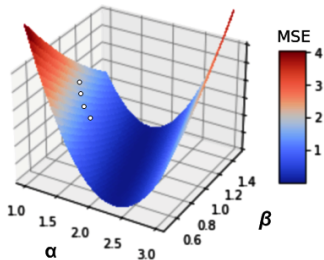
Busca de um erro mínimo (ou mínimo para a Função de Perda)



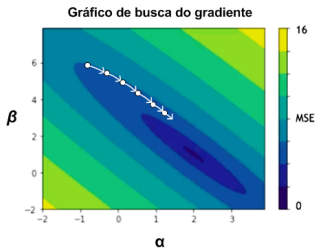
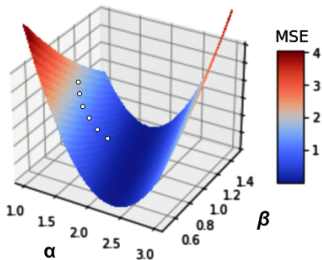
Busca de um erro mínimo (ou mínimo para a Função de Perda)



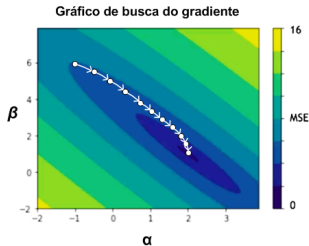
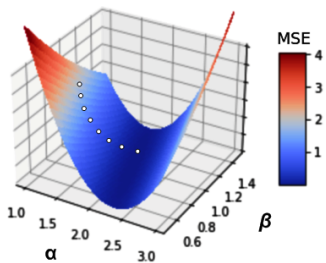
Busca de um erro mínimo (ou mínimo para a Função de Perda)



Busca de um erro mínimo (ou mínimo para a Função de Perda)



Busca de um erro mínimo (ou mínimo para a Função de Perda)



$$\text{Função de Perda } \zeta = \frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2$$

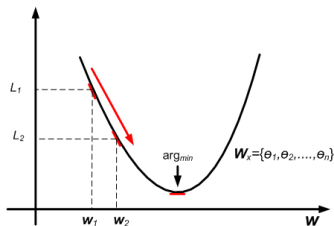


Ao minimizar uma Função de Perda estabelece-se um loop

- ✓ Processo de buscar um mínimo é chamado Gradiente Descendente;
- ✓ O quanto cada passo decrementa é chamado taxa de aprendizagem λ ;
- ✓ Uma época é um atualização do modelo feito sobre todo o dataset;
- ✓ Um batch é uma amostra de todo o dataset;
- ✓ Um step é um update para os pesos (α, β) dos parâmetros



Busca de um mínimo

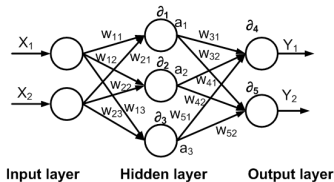


- ✓ O Stochastic Gradient Descent (SGD) é um processo recursivo para a busca de uma função de perda mínima;
- ✓ Na verdade este mínimo é o cálculo de uma derivada ao longo de uma curva;
- ✓ Este processo na verdade é teórico pois o que acontece numa Rede Neural é muito semelhante a esta abstração;



Busca de um mínimo

Estrutura de uma rede neural



Funções de ativação

ReLU
$$f(x) = \begin{cases} 0 & \text{se } x < 0 \\ x & \text{se } x \geq 0 \end{cases}$$

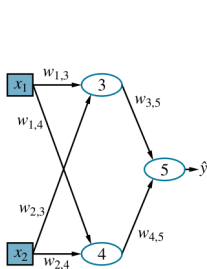
Sigmoid
$$f(x) = \frac{1}{1 + e^{-x}}$$

Softmax
$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

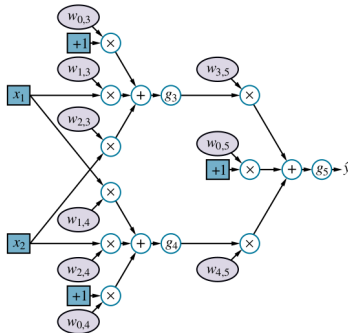
para $j = 1, \dots, K$

- ✓ Cada neurônio fora a entrada tem associado um peso das arestas w e uma função de ativação δ ;
- ✓ Esta função de ativação é uma função diferente de uma equação da reta;
 - ReLU, Sigmoid e Softmax
- ✓ Estas função de ativação tem a finalidade adicionar uma não linearidade ao próximo layer;





(a)



(b)

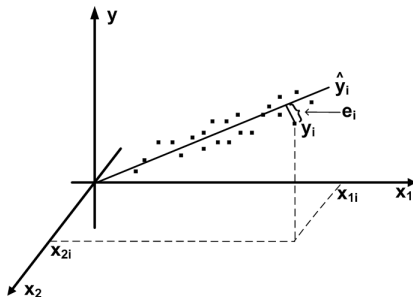
$$\vec{y} = g_5(w_{0,5} + w_{3,5}g_3(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) + w_{4,5}g_4(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2))$$

Processo recursivo para minimizar $\mathcal{L} = \frac{1}{n} \sum_1^n (y - \vec{y})^2$

Otimização da Rede Neural

$$y = g_5(w_{0,5} + w_{3,5}g_3(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) + w_{4,5}g_4(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2))$$

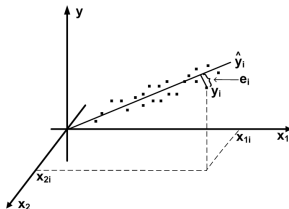
$$y = g_5(\theta_9 + \theta_7g_3(\theta_5 + \theta_1x_1 + \theta_3x_2) + \theta_8g_4(\theta_6 + \theta_2x_1 + \theta_4x_2))$$



[Russel,2021]



Otimização da Rede Neural

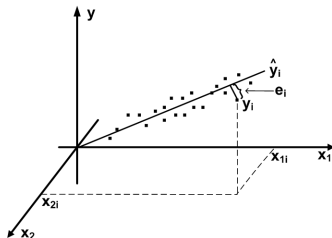


$$\underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}}_{\mathbf{Y}_{(n,1)}} = \underbrace{\begin{pmatrix} x_{11} & x_{12} \\ x_{12} & x_{22} \\ \vdots & \vdots \\ x_{1n} & x_{2n} \end{pmatrix}}_{\underline{X}_{(n,p)}} \cdot \underbrace{\begin{pmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{pmatrix}}_{\boldsymbol{\theta}_{(p,1)}} + \underbrace{\begin{pmatrix} \theta_{k1} \\ \theta_{k2} \\ \vdots \\ \theta_{kn} \end{pmatrix}}_{\boldsymbol{\theta}_{(k,1)}}$$

[Bishop, 2009], [Russel,2021]



Target: Minimizar uma Loss Function (L)

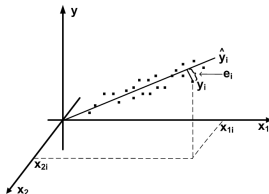


$$\mathbf{y} = h_{\theta}(x) = \theta_1 x + \theta_2$$

$$L(H_{\theta}) = \sum_{j=1}^N (y_i - \mathbf{y})^2 = \sum_{j=1}^N (y_i - \theta_1 x - \theta_2)^2$$

Target: Minimizar uma Loss Function (L)

$$\arg_{\min} L(H_{\theta}) = \sum_{j=1}^N (y_i - \theta_1 x - \theta_2)^2$$

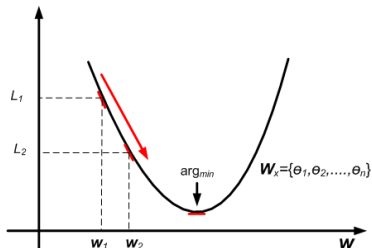


Para calcular a derivada com respeito aos parâmetros θ .

$$\frac{\partial}{\partial \theta_1} \sum_{j=1}^N (y_i - \theta_1 x - \theta_2)^2 = 0$$

$$\frac{\partial}{\partial \theta_2} \sum_{j=1}^N (y_i - \theta_1 x - \theta_2)^2 = 0$$

Target: Minimizar uma Loss Function (L)



$w \leftarrow$ qualquer ponto no parâmetro de espaços

while not converged do

for each w_i in w do

$$w_i \leftarrow w_{i-1} - \alpha \frac{\partial}{\partial w_i} L(w)$$

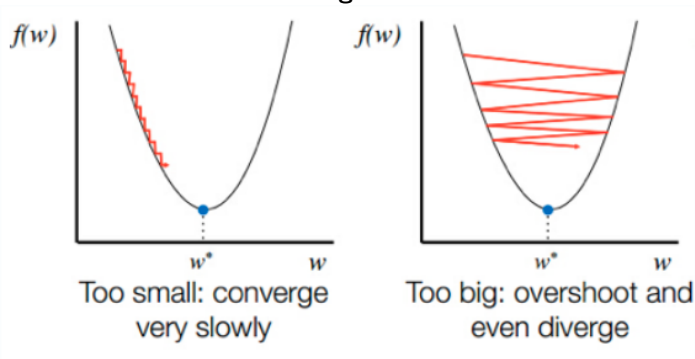
α = Learning rate.

[Bishop, 2009], [Russel,2021]



Target: Minimizar uma Loss Function (L)

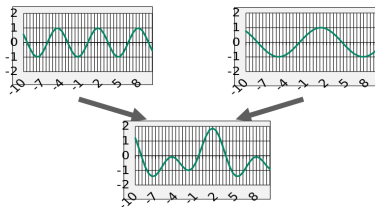
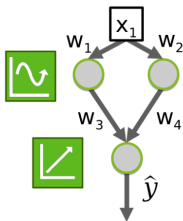
Efeito da escolha do Learning rate.



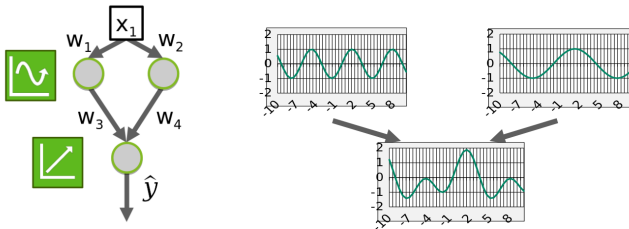
[Bishop, 2009], [Russel,2021]



Qual o papel da função de ativação?



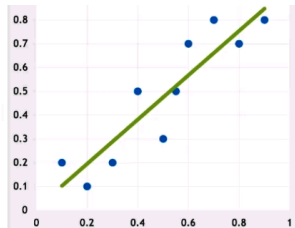
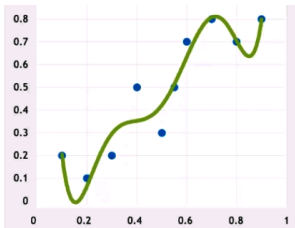
Qual o papel da função de ativação?



Adicionar uma não linearidade ao modelo.

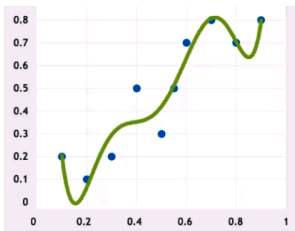
Qual destas linhas de tendência é melhor?

Como escolher entre uma ou outra? Qual métrica podemos utilizar?

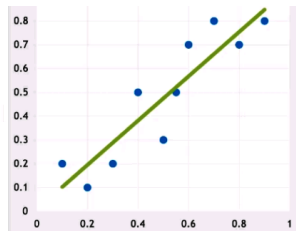


Qual destas linhas de tendência é melhor?

Como escolher entre uma ou outra? Através do RMSE.



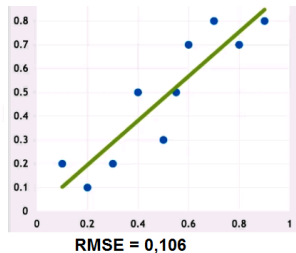
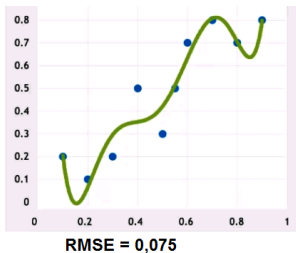
RMSE = 0,075



RMSE = 0,106

Qual destas linhas de tendência é melhor?

Como escolher entre uma ou outra?

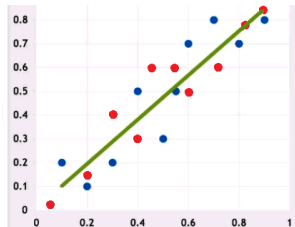
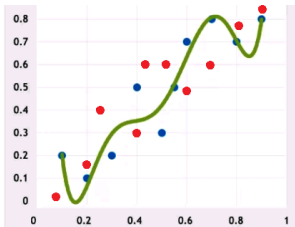


Mas será que só isto é suficiente?



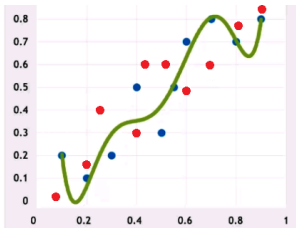
Qual destas linhas de tendência é melhor?

Que tal adicionar mais dados e comparar o RMSE novamente.

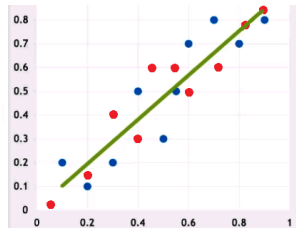


Qual destas linhas de tendência é melhor?

Que tal adicionar mais dados e comparar o RMSE novamente.



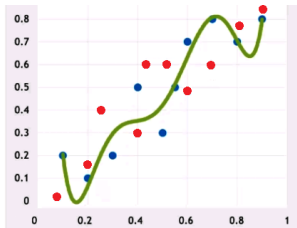
RMSE = 0,1311



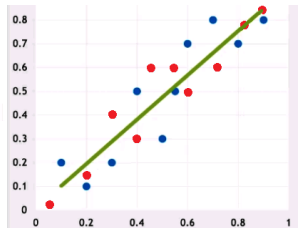
RMSE = 0,0787

Qual destas linhas de tendência é melhor?

Que tal adicionar mais dados e comparar o RMSE novamente.



RMSE = 0,1311



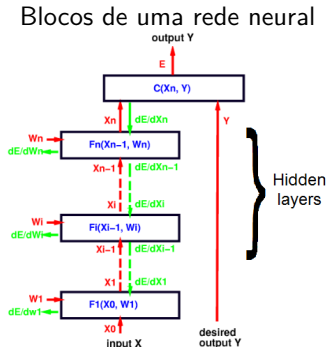
RMSE = 0,0787

A primeira curva é ajustada para os dados e não generaliza bem o modelo.

Isto é o que se chama de **Overfitting**.



Busca de um mínimo - Propagação direta (feed-forward)



- ✓ Aprendizados de máquina complexos podem ser construídos montando módulos em redes;
 - Ex: arquitetura feed-forward sequencial em camadas (cascata)
- ✓ Cada módulo é um objeto;
- ✓ Método feed-forward:

Tomando $X = X_0$

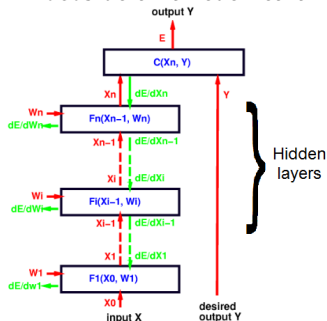
$$X_i = F_i(X_{i-1}, W_i) \quad \forall i \in [1, n]$$

$$E(Y, X, W) = C(X_n, Y)$$



Busca de um mínimo - Cálculo do Gradiente

Blocos de uma rede neural



- ✓ Para treinar um sistema multi-módulo, devemos calcular o gradiente de E em relação a todos os parâmetros do sistema (todos os W_i).
- ✓ Vamos considerar o módulo I cujo método feed-forward calcula $X_i = F_i(X_{i-1}, W_i)$.
- ✓ Aplica-se a regra da cadeia para determinar a influência de cada componente do peso W_i sobre E ;

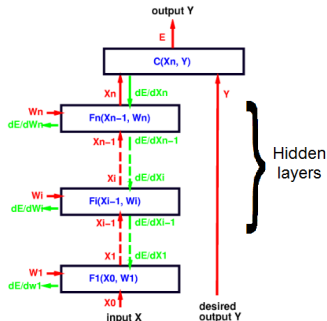
Tomando $\frac{\partial E}{\partial W_i}$

$$\frac{\partial E}{\partial W_i} = \frac{\partial E}{\partial X_i} \frac{\partial F_i(X_{i-1}, W_i)}{\partial W_i}$$



Busca de um mínimo - Cálculo do Gradiente

Blocos de uma rede neural



$$\frac{\partial E}{\partial W_i} = \frac{\partial E}{\partial X_i} \frac{\partial F_i(X_{i-1}, W_i)}{\partial W_i}$$

$\frac{\partial F_i(X_{i-1}, W_i)}{\partial W_i}$ é o Jacobiano de F_i com respeito a W_i .

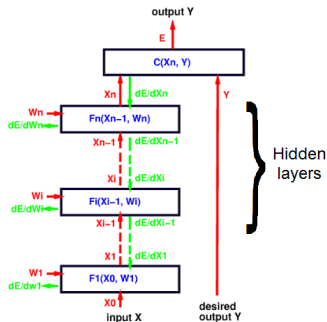
$$\left| \frac{\partial F_i(X_{i-1}, W_i)}{\partial W_i} \right|_{kl} = \frac{\partial [F_i(X_{i-1}, W_i)]_k}{\partial [W_i]_l}$$

(k, l) elemento do Jacobiano indica quanto do peso da k -ésima saída tem influência quando pondera-se o l -ésimo peso;



Busca de um mínimo - Back-propagation

Para calcular todas as derivadas, usamos uma varredura reversa chamada algoritmo de retropropagação (back-propagation) que usa a equação de recorrência para $\frac{\partial E}{\partial W_i}$.



$$\frac{\partial E}{\partial X_n} = \frac{\partial C(X_n, Y)}{\partial X_n}$$

$$\frac{\partial E}{\partial W_{n-1}} = \frac{\partial E}{\partial X_n} \frac{\partial F_n(X_{n-1}, W_n)}{\partial W_{n-1}}$$

$$\frac{\partial E}{\partial W_n} = \frac{\partial E}{\partial X_n} \frac{\partial F_n(X_{n-1}, W_n)}{\partial W_n}$$

$$\frac{\partial E}{\partial W_{n-2}} = \frac{\partial E}{\partial X_{n-1}} \frac{\partial F_{n-1}(X_{n-2}, W_{n-1})}{\partial W_{n-2}}$$

$$\frac{\partial E}{\partial W_{n-1}} = \frac{\partial E}{\partial X_{n-1}} \frac{\partial F_{n-1}(X_{n-2}, W_{n-1})}{\partial W_{n-1}}$$

... até chegar ao primeiro nível onde

Tem-se todo o $\frac{\partial E}{\partial W_i}$ para $i \in [1, n]$



Variantes do SGD e suas características

Variante	Descrição	Características
Vanila GD	Computa o gradiente com o dataset inteiro de uma única vez. $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta)$	Computação redundante. Lento. Intratável se a memória é menor que o dataset inteiro. Garante convergência para um mínimo global.
SGD	Computa o gradiente com respeito a θ para cada treinamento com exemplo $x^{(i)}$ e label $y^{(i)}$. $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$	Evita redundância. Mais rápido que o Vanila. Tem muitos updates e convergência flutua ao redor de um mínimo local.
Mini-batch GD	Combina os dois primeiros. Computa o gradiente para cada mini-batch de n amostras de treinamento. $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)})$	Reduz a variância dos updates dos parâmetros. Faz uso de matriz de parâmetros altamente otimizada. Tamanho de mini-batch típico 32 a 64.



Otimização do SGD

- ✓ SGD tem problemas em áreas onde a curva da superfície é mais acentuada em uma dimensão do que em outra (ótimo local).
- ✓ O SGD oscila ao longo de encostas e faz um progresso hesitante em direção ao ótimo local.

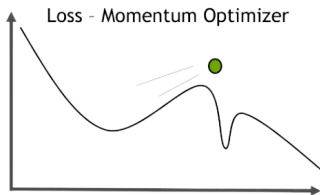


Otimização do SGD

- ✓ SGD tem problemas em áreas onde a curva da superfície é mais acentuada em uma dimensão do que em outra (ótimo local).
- ✓ O SGD oscila ao longo de encostas e faz um progresso hesitante em direção ao ótimo local.

A solução é aplicar o Momentum que acelera uma fração de γ do vetor de updates.

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$



Outras otimizações

- Adam
- Adagrad
- RMSprop



Conjunto de dados

Treinamento	Validação	Teste
Uma amostra do dataset usado para ajustar o modelo da rede neural.	Uma amostra dos dados usados para prover uma avaliação sem viés do ajuste do modelo enquanto ajusta-se os parâmetros.	Uma amostra dos dados usada para fornecer uma avaliação sem viés após o modelo ajustado.
80% da amostra dos dados	20% da amostra dos dados	Usado uma vez após o modelo estar completamente treinado.
Usa-se cross-validation		Conjunto de dados bem curado.

Método de Validação Cruzada



Ajuste dos dados

- ✓ Uma parte importante é fazer uma avaliação dos dados a serem usados.
- ✓ A escala e o tamanho dos dados possivelmente devem ser ajustados antes de serem processados.
- ✓ Verificar se os dados contém rótulos bem definidos e se não existem dados com ruído.
 - Deve-se aplicar filtros para limpar os dados.
- ✓ Inicialmente os dados devem ser convertidos para uma série entre $[0, 1]$ ou $[-1, +1]$.
 - Ao se fazer isto antes do treinamento a normalização se perde ao longo do treinamento.
- ✓ Usar a normalização dentro do batch por sua vez elimina o problema anterior e diminui o número de épocas.



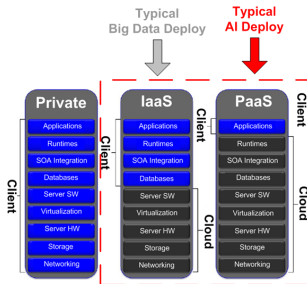
1. Abordagens para Extração com Big Data
2. Frameworks
3. Aplicações Típicas
4. Custos de Computação
5. Introdução da Programação MapReduce com Hadoop
6. Introdução à IA
7. Mecanismo de Computação
8. Adicionando Valor com IA



Big Data e IA são técnicas convergentes.

Plataforma Possível

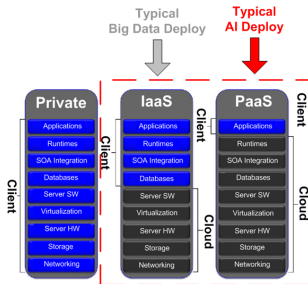
- ✓ IA é um implementação em PaaS;
- ✓ Big datacenters (**GPU Clusters**);



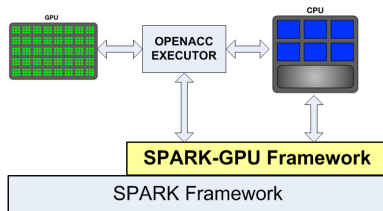
Big Data e IA são técnicas convergentes.

Plataforma Possível

- ✓ IA é um implementação em PaaS;
- ✓ Big datacenters (**GPU Clusters**);



Big Data integrado com IA.

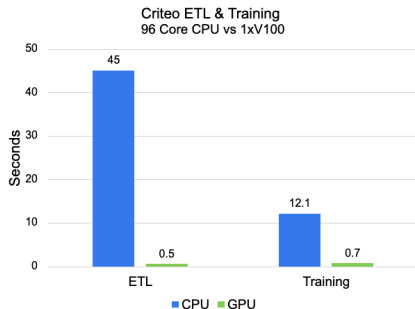
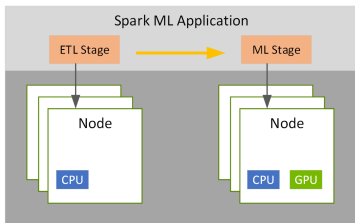


Aplicações do tipo ETL = Extracting, Transforming, and Loading data.



Exemplos de convergência de frameworks Big Data e IA

- ✓ 1TB de Click dataset de Logs;
- ✓ Spark provêm maior ingestão de dados;

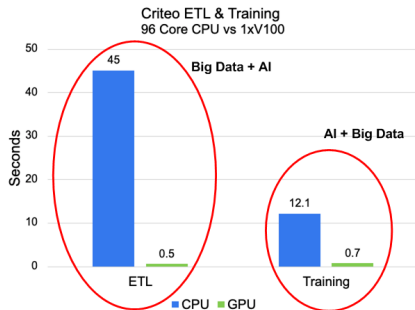
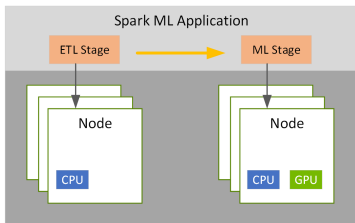


Fonte: <https://github.com/NVIDIA/spark-rapids-examples>



Exemplos de convergência de frameworks Big Data e IA

- ✓ 1TB de Click dataset de Logs;
- ✓ Spark provêm maior ingestão de dados;

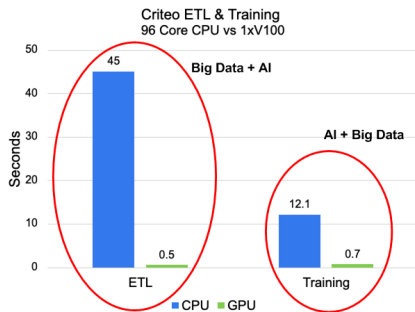
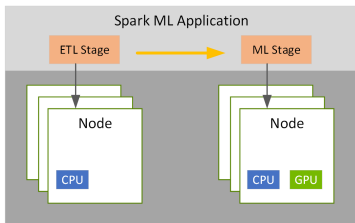


Fonte: <https://github.com/NVIDIA/spark-rapids-examples>



Exemplos de convergência de frameworks Big Data e IA

- ✓ 1TB de Click dataset de Logs;
- ✓ Spark provém maior ingestão de dados;



Fonte: <https://github.com/NVIDIA/spark-rapids-examples>

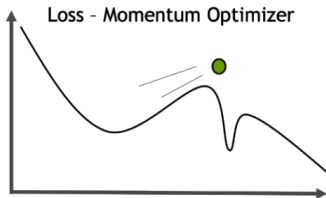
Como podemos solucionar o problemas de latência em IoT?



Inteligência para Edge Computing

Computação do SGD

$$\theta_{t+1} = \theta_t - \eta g_t;$$
$$g_t = \frac{1}{N} \sum_{i=1}^n \nabla_{\theta} \mathcal{L}(\theta, x_{i:i+n})$$



(a.)

Figura 1: Processo de computação do Gradient Descent

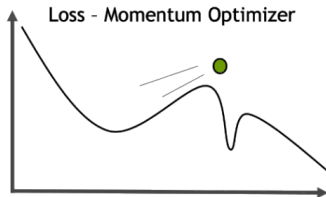


Inteligência para Edge Computing

Computação do SGD

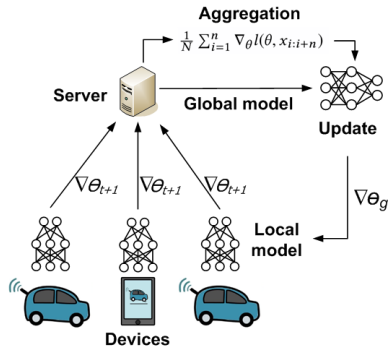
$$\theta_{t+1} = \theta_t - \eta g_t;$$

$$g_t = \frac{1}{N} \sum_{i=1}^n \nabla_{\theta} \mathcal{L}(\theta, x_{i:i+n})$$



(a.)

Federated Learning



(b.)

Figura 1: Processo de computação do Gradient Descent



1. Abordagens para Extração com Big Data
2. Frameworks
 - MapReduce
 - Streaming Processing
3. Aplicações Típicas
4. Custos de Computação
 - Complexidade do Modelo MapReduce
5. Introdução da Programação MapReduce com Hadoop
 - Entrada e Saída de Dados
 - Programação MapReduce com o Hadoop
6. Introdução à IA
7. Mecanismo de Computação
 - Cálculo do SGD
 - Função de Ativação
 - Overfitting
 - Feed-forward
 - Back-propagation
 - Otimização SGD
 - Conjunto de Dados para a Computação





ASSUNCAO, M. D. et al. Big Data computing and clouds: Trends and future directions. *Journal of Parallel and Distributed Computing*, Elsevier Science Publishers Ltd., v. 79–80, p. 3–15, 2015. ISSN 0743-7315.



DEAN, J.; GHEMAWAT, S. MapReduce - Simplified Data Processing on Large Clusters. In: *OSDI*. [S.l.: s.n.], 2004. p. 137–150.



DEAN, J.; GHEMAWAT, S. MapReduce - A Flexible Data Processing Tool. *Communications of the ACM*, ACM, New York, NY, USA, v. 53, n. 1, p. 72–77, 2010. ISSN 0001-0782.



LAKSHMAN, A.; MALIK, P. Cassandra: a decentralized structured storage system. *SIGOPS Oper. Syst. Rev.*, ACM, New York, NY, USA, v. 44, n. 2, p. 35–40, Apr 2010. ISSN 0163-5980. Disponível em: <<http://doi.acm.org/10.1145/1773912.1773922>>.



LESKOVEC, J.; RAJARAMAN, A.; ULLMAN, J. D. *Mining of Massive Datasets*. 3rd. ed. [S.l.]: Cambridge University Press, 2020. 467 p. ISBN 978-1-107-07723-2.



MATTEUSSI, K. J. et al. Performance Evaluation Analysis of Spark Streaming Backpressure for Data-Intensive Pipelines. *Sensors*, MDPI, v. 22, n. 13, p. 4756(1–28), jun. 2022. ISSN 1424-8220.



VENNER, J. *Pro Hadoop - Build Scalable, Distributed Applications in the Cloud*. 1st. ed. [S.l.]: Apress, Inc., 2009. ISBN 9781430219422.



WHITE, T. *Hadoop - The Definitive Guide*. 3rd. ed. California, USA: O'Reilly Media, Inc., 2012. 688 p. ISBN 978-1-4493-1152-0.





EXTRAÇÃO DE DADOS POR BIG DATA E CONVERGÊNCIA COM IA

Prof. Dr. Julio César Santos dos Anjos
Dados e Informação

Técnicas de Big Data e Convergência com IA

— 23 de maio de 2025